RM0468
Reference manual

STM32H723/733, STM32H725/735 and STM32H730 Value line advanced Arm®-based 32-bit MCUs

Introduction

This reference manual targets application developers. It provides complete information on how to use the STM32H723/733, STM32H725/735 and STM32H730 memory and peripherals.

The STM32H723/733, STM32H725/735 and STM32H730 are lines of microcontrollers with different memory sizes, packages and peripherals.They are referred to as STM32H72x and STM32H73x hereafter.

The devices include ST state-of-the-art patented technology.

For ordering information, mechanical, and electrical device characteristics refer to the corresponding datasheets.

For information on the Arm® Cortex®-M7 with FPU core, refer to the corresponding Arm *Technical Reference Manuals*.

Related documents

- Arm® Cortex®-M7 technical reference manual, available from www.arm.com
- Cortex®-M7 programming manual (PM0253)
- STM32H723xx, STM32H733xx, STM32H725xx, STM32H730xx, and STM32H735xx datasheets
- STM32H72xx/73xx errata sheet.

Table 7. Register boundary addresses(1) (continued)

Boundary address	Peripheral	Bus	Register map
0x5200BC00 - 0x5200BFFF	OTFDEC2	AHB3 (D1)	*Section 42.6: OTFDEC registers*
0x5200B800 - 0x5200BBFF	OTFDEC1		*Section 42.6: OTFDEC registers*
0x5200B400 - 0x5200B7FF	OCTOSPI I/O		*Section 26.5: OCTOSPIM registers*
0x5200B000 - 0x5200B3FF	Delay Block OCTOSPI2		*Section 27.4: DLYB registers*
0x5200A000 - 0x5200AFFF	OCTOSPI2		*Section 25.7: OCTOSPI registers*
0x52009000 - 0x520093FF	RAMECC D1 domain		*Section 3.4: RAMECC registers*
0x52008000 - 0x52008FFF	Delay Block SDMMC1		*Section 27.4: DLYB registers*
0x52007000 - 0x52007FFF	SDMMC1		*Section 60.10: SDMMC registers*
0x52006000 - 0x52006FFF	Delay Block OCTOSPI1		*Section 27.4: DLYB registers*
0x52005000 - 0x52005FFF	OCTOSPI1 control registers		*Section 25.7: OCTOSPI registers*
0x52004000 - 0x52004FFF	FMC control registers		*Section 24.7.6: NOR/PSRAM controller registers*, *Section 24.8.7: NAND Flash controller registers*, *Section 24.9.5: SDRAM controller registers*
0x52002000 - 0x52002FFF	Flash interface registers		*Section 4.9: FLASH registers*
0x52001000 - 0x52001FFF	Chrom-Art (DMA2D)		*Section 18.5: DMA2D registers*
0x52000000 - 0x52000FFF	MDMA		*Section 14.5: MDMA registers*
0x51000000 - 0x510FFFFF	GPV		*Section 2.2: AXI interconnect matrix (AXIM)*
0x50003000 - 0x50003FFF	WWDG	APB3 (D1)	*Section 49.5: WWDG interrupts*
0x50001000 - 0x50001FFF	LTDC		*Section 38.7: LTDC registers*
0x50000000 - 0x50000FFF	Reserved		-
0x48024400 - 0x48024 7FFF	CORDIC	AHB2 (D2)	*Section 22.4: CORDIC registers*
0x48024000 - 0x480243FF	FMAC		*Section 23.4: FMAC registers*
0x48023000 - 0x48023FFF	RAMECC D2 domain		*Section 3.4: RAMECC registers*
0x48022800 - 0x48022BFF	Delay Block SDMMC2		*Section 27.4: DLYB registers*
0x48022400 - 0x480227FF	SDMMC2		*Section 60.10: SDMMC registers*
0x48021800 - 0x48021BFF	RNG		*Section 39.7: RNG registers*
0x48021400 - 0x480217FF	HASH		*Section 41.7: HASH registers*
0x48021000 - 0x480213FF	CRYPTO		*Section 40.7: CRYP registers*
0x48020400 - 0x480207FF	PSSI		*Section 37.5: PSSI registers*
0x48020000 - 0x480203FF	DCMI		*Section 36.5: DCMI registers*

Table 7. Register boundary addresses(1) (continued)

Boundary address	Peripheral	Bus	Register map
0x40040000 - 0x4007FFFF	USB1 OTG_HS	AHB1 (D2)	*Section 62.14: OTG_HS registers*
0x40028000 - 0x400293FF	ETHERNET MAC		*Section 63.11: Ethernet registers*
0x40024400 - 0x400247FF	Reserved		Reserved
0x40022000 - 0x400223FF	ADC1 - ADC2		*Section 28.7: ADC common registers*
0x40020800 - 0x40020BFF	DMAMUX1		*Section 17.6: DMAMUX registers*
0x40020400 - 0x400207FF	DMA2		*Section 15.5: DMA registers*
0x40020000 - 0x400203FF	DMA1		*Section 15.5: DMA registers*
0x40017800 - 0x40017FFF	DFSDM1	APB2 (D2)	*Section 35.7: DFSDM channel y registers (y=0..7)*, *Section 35.8: DFSDM filter x module registers (x=0..3)*
0x40015800 - 0x40015BFF	SAI1		*Section 56.6: SAI registers*
0x40015000 - 0x400153FF	SPI5		*Section 55.11: SPI/I2S registers*
0x40014800 - 0x40014BFF	TIM17		*Section 46.6: TIM16/TIM17 registers*
0x40014400 - 0x400147FF	TIM16		*Section 46.6: TIM16/TIM17 registers*
0x40014000 - 0x400143FF	TIM15		*Section 46.5: TIM15 registers*
0x40013400 - 0x400137FF	SPI4		*Section 55.11: SPI/I2S registers*
0x40013000 - 0x400133FF	SPI1 / I2S1		*Section 55.11: SPI/I2S registers*
0x40011C00 - 0x40011FFF	USART10		*Section 53.8: USART registers*
0x40011800 - 0x40011BFF	UART9		*Section 53.8: USART registers*
0x40011400 - 0x400117FF	USART6		*Section 53.8: USART registers*
0x40011000 - 0x400113FF	USART1		*Section 53.8: USART registers*
0x40010400 - 0x400107FF	TIM8		*Section 43.4: TIM1/TIM8 registers*
0x40010000 - 0x400103FF	TIM1		*Section 43.4: TIM1/TIM8 registers*

Table 7. Register boundary addresses(1) (continued)

Boundary address	Peripheral	Bus	Register map
0x4000E400 - 0x4000E7FF	TIM24	APB1 (D2)	*Section 44.4: TIM2/TIM3/TIM4/TIM5/TIM23/TIM24 registers*
0x4000E000 - 0x4000E3FF	TIM23		*Section 44.4: TIM2/TIM3/TIM4/TIM5/TIM23/TIM24 registers*
0x4000D400 - 0x4000D7FF	FDCAN3		*Section 61.5: FDCAN registers*
0x4000AC00 - 0x4000D3FF	CAN Message RAM		*Section 61.5: FDCAN registers*
0x4000A800 - 0x4000ABFF	CAN CCU		*Section 61.5: FDCAN registers*
0x4000A400 - 0x4000A7FF	FDCAN2		*Section 61.5: FDCAN registers*
0x4000A000 - 0x4000A3FF	FDCAN1		*Section 61.5: FDCAN registers*
0x40009400 - 0x400097FF	MDIOS		*Section 59.4: MDIOS registers*
0x40009000 - 0x400093FF	OPAMP		*Section 34.6: OPAMP registers*
0x40008800 - 0x40008BFF	SWPMI		*Section 58.6: SWPMI registers*
0x40008400 - 0x400087FF	CRS		*Section 9.8: CRS registers*
0x40007C00 - 0x40007FFF	UART8		*Section 53.8: USART registers*
0x40007800 - 0x40007BFF	UART7		*Section 53.8: USART registers*
0x40007400 - 0x400077FF	DAC1/2		*Section 31.7: DAC registers*
0x40006C00 - 0x40006FFF	HDMI-CEC		*Section 64.7: HDMI-CEC registers*
0x40006400 - 0x400067FF	I2C5		*Section 52.7: I2C registers*
0x40005C00 - 0x40005FFF	I2C3		*Section 52.7: I2C registers*
0x40005800 - 0x40005BFF	I2C2		*Section 52.7: I2C registers*
0x40005400 - 0x400057FF	I2C1		*Section 52.7: I2C registers*
0x40005000 - 0x400053FF	UART5		*Section 53.8: USART registers*
0x40004C00 - 0x40004FFF	UART4		*Section 53.8: USART registers*
0x40004800 - 0x40004BFF	USART3		*Section 53.8: USART registers*
0x40004400 - 0x400047FF	USART2		*Section 53.8: USART registers*
0x40004000 - 0x400043FF	SPDIFRX1		*Section 57.5: SPDIFRX interface registers*
0x40003C00 - 0x40003FFF	SPI3 / I2S3		*Section 55.11: SPI/I2S registers*
0x40003800 - 0x40003BFF	SPI2 / I2S2		*Section 55.11: SPI/I2S registers*
0x40002C00 - 0x40002FFF	Reserved		Reserved
0x40002400 - 0x400027FF	LPTIM1		*Section 48.7: LPTIM registers*
0x40002000 - 0x400023FF	TIM14		*Section 44.4: TIM2/TIM3/TIM4/TIM5/TIM23/TIM24 registers*
0x40001C00 - 0x40001FFF	TIM13		*Section 44.4: TIM2/TIM3/TIM4/TIM5/TIM23/TIM24 registers*
0x40001800 - 0x40001BFF	TIM12		*Section 44.4: TIM2/TIM3/TIM4/TIM5/TIM23/TIM24 registers*

Table 16 shows the correspondence between the number of wait states (LATENCY), the programming delay parameter (WRHIGHFREQ), the embedded Flash memory clock frequency and its supply voltage ranges.

Table 16. FLASH recommended number of wait states and programming delay

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;35 MHz]	[0 MHz;50 MHz]	[0 MHz;67 MHz]	[0 MHz;70 MHz]
1 WS (2 FLASH clock cycles)	01	]35 MHz;70 MHz]	]50 MHz;100 MHz]	]67 MHz;133 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	10	]70 MHz;85 MHz]	]100 MHz;150 MHz]	]133 MHz;200 MHz]	]140 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	11	-	-	-	]210 MHz;275MHz]

Adjusting system frequency

After power-on, a default 7 wait-state latency is specified in FLASH_ACR register, in order to accommodate AXI interface clock frequencies with a safety margin (see *Table 16*).

When changing the AXI bus frequency, the application software must follow the below sequence in order to tune the number of wait states required to access the non-volatile memory.

To increase the embedded Flash memory clock source frequency:

1. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in the FLASH_ACR register, as described in *Table 16*.
2. Check that the new number of wait states is taken into account by reading back the FLASH_ACR register.
3. Modify the embedded Flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of the reset and clock controller (RCC).
4. Check that the new embedded Flash memory clock source and/or the new AXI bus clock prescaler value are taken in account by reading back the embedded Flash memory clock source status and/or the AXI bus prescaler value in the RCC_CFGR register of the reset and clock controller (RCC).

To decrease the embedded Flash memory clock source frequency:

1. Modify the embedded Flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of reset and clock controller (RCC).
2. Check that the embedded Flash memory new clock source and/or the new AXI bus clock prescaler value are taken into account by reading back the embedded Flash

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **IO_HSLV**: I/O high-speed at low-voltage configuration bit
This bit indicates that the product operates below 2.5 V.
0: Product working in the full voltage range, I/O speed optimization at low-voltage disabled
1: Product operating below 2.5 V, I/O speed optimization at low-voltage feature allowed

Bits 28:26 Reserved, must be kept at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 **SECURITY**: Security enable option configuration bit
The SECURITY option bit enables the secure access mode of the device during an option byte change. The change will be taken into account at next power-on reset. Once it is enabled, the security feature can be disabled if no areas are protected by PCROP or Secure access mode. If there are secure-only or PCROP protected areas, perform a regression level (from level 1 to 0) and set all the bits to unprotect secure-only areas and PCROP areas.
0: Security feature disabled
1: Security feature enabled.

Bits 20:19 **ST_RAM_SIZE[1:0]**: ST RAM size option configuration bits
00: 2 Kbytes reserved to ST code
01: 4 Kbytes reserved to ST code
10: 8 Kbytes reserved to ST code
11: 16 Kbytes reserved to ST code
Note: This bitfield is effective only when the security is enabled (SECURITY = 1).
The whole DTCM RAM is always available for the application whatever ST_RAM_SIZE option byte configuration.

Bit 18 **IWDG_FZ_SDBY**: IWDG Standby mode freeze option configuration bit
This option bit is used to freeze or not the independent watchdog IWDG1 in system Standby mode.
0: Independent watchdog frozen in Standby mode
1: Independent watchdog keep running in Standby mode.

Bit 17 **IWDG_FZ_STOP**: IWDG Stop mode freeze option configuration bit
This option bit is used to freeze or not the independent watchdog IWDG1 in system Stop mode.
0: Independent watchdog frozen in system Stop mode
1: Independent watchdog keep running in system Stop mode.

Bit 16 Reserved, must be kept at reset value.

Bits 15:8 **RDP[7:0]**: Readout protection level option configuration bits
RDP bits are used to change the readout protection level. This change is possible only when the current protection level is different from level 2. The possible configurations are:
0xAA: global readout protection level 0
0xCC: global readout protection level 2
others values: global readout protection level 1.

Bit 7 **NRST_STDY_D1**: D1 domain DStandby entry reset option configuration bit
0: a reset is generated when entering DStandby mode on D1 domain.
1: no reset generated when entering DStandby mode on D1 domain

Bit 6 **NRST_STOP_D1**: D1 domain DStop entry reset option configuration bit
0: a reset is generated when entering DStop mode on D1 domain.
1: no reset generated when entering DStop mode on D1 domain.

Boot from pin reset

When a pin reset occurs, V_{DD} is still present. As a results:

- The regulator settling time is faster since the reference voltage is already stable.
- The HSI restart delay may be needed if the HSI was not enabled when the NRST occurred, otherwise this restart delay phase is skipped.
- The Flash memory power recovery delay can also be skipped if the Flash memory was enabled when the NRST occurred.

Note: The boot sequence is similar for pwr_bor_rst, lpwr_rst, STFxRESET, iwdg1_out_rst and wwdg1_out_rst (if WW1RSC = 1).

Boot from system Standby

When waking up from system Standby, the reference voltage is stable since V_{DD} has not been removed. As a result, the regulator settling time is fast. Since V_{CORE} was not present, the restart delay for the HSI, the Flash memory power recovery and the option byte reloading cannot be skipped.

Restart from system Stop

When restarting from system Stop, V_{DD} is still present. As a result, the sequence is mainly composed of two steps:

1. Regulator settling time to reach VOS3 (default voltage)
2. HSI/CSI restart delay. This step can be skipped if HSIKERON or CSIKERON bit, in *RCC source control register (RCC_CR)* is set to 1.

Boot from domain DStandby

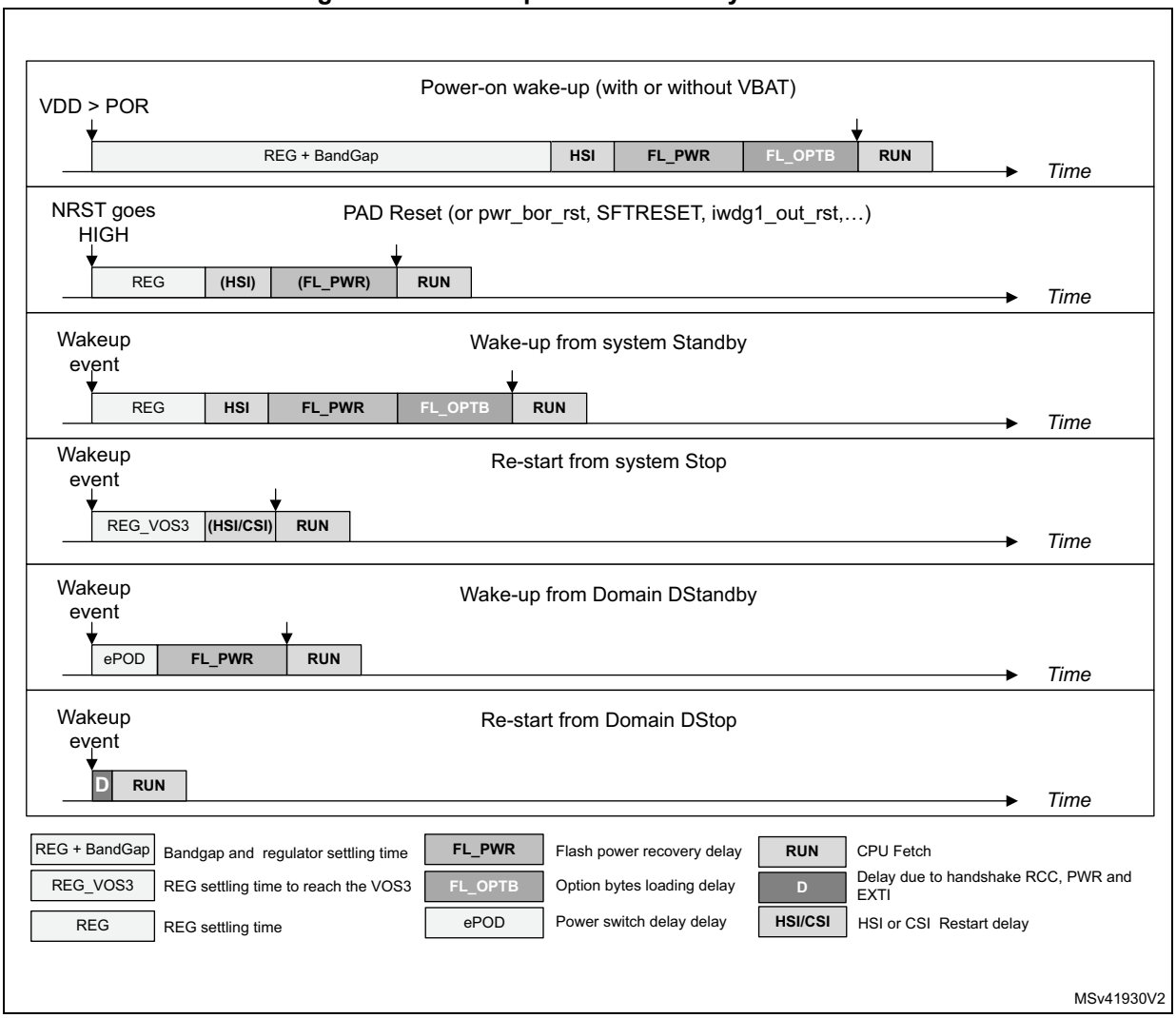
The boot sequence of a domain from domain DStandby is mainly composed of two steps:

1. The power switch settling time (the regulator is already activated).
2. The Flash memory power recovery.

Restart from domain DStop

The restart sequence of a domain from domain DStop is mainly composed of the handshake between the RCC, EXTI and PWR blocks.

Figure 44. Boot sequences versus system states



8.5 RCC clock block functional description

The RCC provides a wide choice of clock generators:

- HSI (High-speed internal oscillator) clock: ~ 8, 16, 32 or 64 MHz
- HSE (High-speed external oscillator) clock: 4 to 48 MHz
- LSE (Low-speed external oscillator) clock: 32 kHz
- LSI (Low-speed internal oscillator) clock: ~ 32 kHz
- CSI (Low-power internal oscillator) clock: ~4 MHz
- HSI48 (High-speed 48 MHz internal oscillator) clock: ~48 MHz

It offers a high flexibility for the application to select the appropriate clock for CPU and peripherals, in particular for peripherals that require a specific clock such as Ethernet, USB OTG-FS and HS, SPI/I2S, SAI and SDMMC.

To optimize the power consumption, each clock source can be switched ON or OFF independently.

The RCC provides up to 3 PLLs; each of them can be configured with integer or fractional ratios.

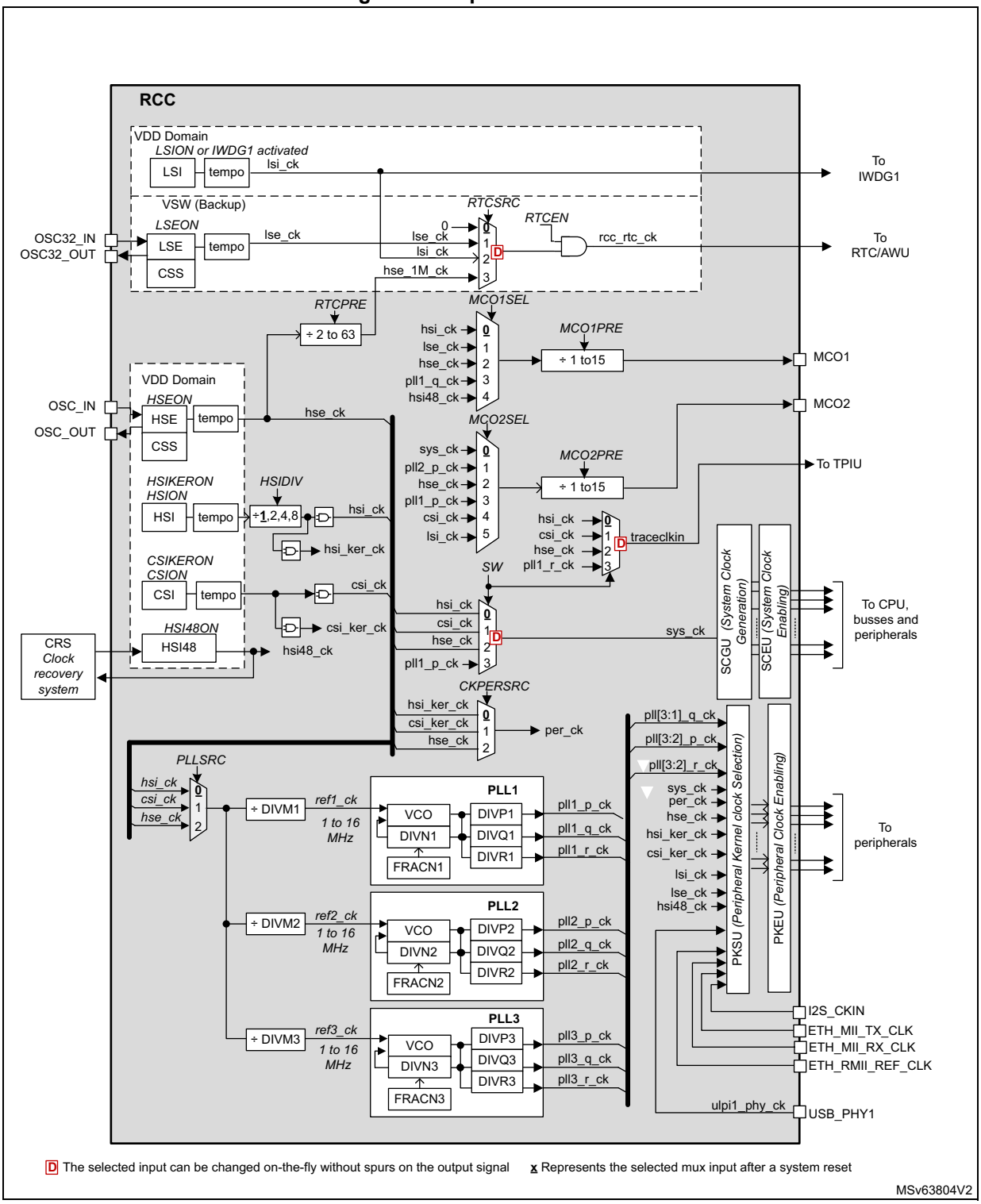
As shown in the *Figure 45*, the RCC offers 2 clock outputs (MCO1 and MCO2), with a great flexibility on the clock selection and frequency adjustment.

The SCGU block (System Clock Generation Unit) contains several prescalers used to configure the CPU and bus matrix clock frequencies.

The PKSU block (Peripheral Kernel clock Selection Unit) provides several dynamic switches allowing a large choice of kernel clock distribution to peripherals.

The PKEU (Peripheral Kernel clock Enable Unit) and SCEU (System Clock Enable Unit) blocks perform the peripheral kernel clock gating, and the bus interface/cores/bus matrix clock gating, respectively.

Figure 45. Top-level clock tree



8.5.1 Clock naming convention

The RCC provides clocks to the complete circuit. To avoid misunderstanding, the following terms are used in this document:

- Peripheral clocks

  The peripheral clocks are the clocks provided by the RCC to the peripherals. Two kinds of clock are available:

  – The bus interface clocks

  – The kernel clocks

  A peripheral receives from the RCC a bus interface clock in order to access its registers, and thus control the peripheral operation. This clock is generally the AHB, APB or AXI clock depending on which bus the peripheral is connected to. Some peripherals only need a bus interface clock (e.g. RNG, TIMx).

  Some peripherals also require a dedicated clock to handle the interface function. This clock is named “kernel clock”. As an example, peripherals such as SAI have to generate specific and accurate master clock frequencies, which require dedicated kernel clock frequencies. Another advantage of decoupling the bus interface clock from the specific interface needs, is that the bus clock can be changed without reprogramming the peripheral.

- CPU clocks

  The CPU clock is the clock provided to the CPU. It is derived from the system clock (**sys_ck**).

- Bus matrix clocks

  The bus matrix clocks are the clocks provided to the different bridges (APB, AHB or AXI). These clocks are derived from the system clock (**sys_ck**).

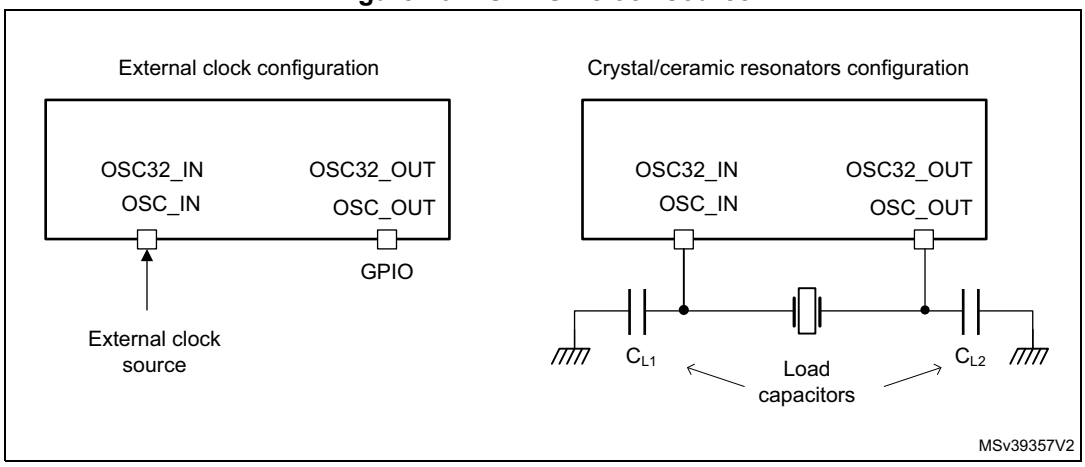
8.5.2 Description of the oscillators

HSE oscillator

The HSE block can generate a clock from two possible sources:

- External crystal/ceramic resonator
- External clock source

Figure 46. HSE/LSE clock source



External clock source (HSE bypass)

In this mode, an external clock source must be provided to OSC_IN pin. This mode is selected by setting the HSEBYP and HSEON bits of the *RCC source control register (RCC_CR)* to 1. The external clock source (square, sinus or triangle) with ~50% duty cycle has to drive the OSC_IN pin.

External crystal/ceramic resonator

The oscillator is enabled by setting the HSEBYP bit to 0 and HSEON bit to 1.

The HSE can be used when the product requires a very accurate high-speed clock.

The associated hardware configuration is shown in *Figure 46*: the resonator and the load capacitors have to be placed as close as possible to the oscillator pins in order to minimize output distortion and startup stabilization time. The loading capacitance values must be adjusted according to the selected crystal or ceramic resonator. Refer to the electrical characteristics section of the datasheet for more details.

The HSERDY flag of the *RCC source control register (RCC_CR)*, indicates whether the HSE oscillator is stable or not. At startup, the **hse_ck** clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the *RCC clock source interrupt enable register (RCC_CIER)*.

The HSE can be switched ON and OFF through the HSEON bit. Note that the HSE cannot be switched OFF if one of the two conditions is met:

- The HSE is used directly (via software mux) as system clock
- The HSE is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSEON bit to 0.

The HSE is automatically disabled by hardware, when the system enters Stop or Standby mode (refer to *Section 8.5.7: Handling clock generators in Stop and Standby mode* for additional information).

In addition, the HSE clock can be driven to the MCO1 and MCO2 outputs and used as clock source for other application components.

LSE oscillator

The LSE block can generate a clock from two possible sources:

- External crystal/ceramic resonator
- External user clock

External clock source (LSE bypass)

In this mode, an external clock source must be provided to OSC32_IN pin. The input clock can have a frequency up to 1 MHz. This mode is selected by setting the LSEBYP and LSEON bits of *RCC backup domain control register (RCC_BDCR)* to 1. The external clock signal (square, sinus or triangle) with ~50% duty cycle has to drive the OSC32_IN pin.

External crystal/ceramic resonator (LSE crystal)

The LSE clock is generated from a 32.768 kHz crystal or ceramic resonator. It has the advantage to provide a low-power highly accurate clock source to the real-time clock (RTC) for clock/calendar or other timing functions.

The LSERDY flag of the *RCC backup domain control register (RCC_BDCR)* indicates whether the LSE crystal is stable or not. At startup, the LSE crystal output clock signal is not released until this bit is set by hardware. An interrupt can be generated if enabled in the *RCC clock source interrupt enable register (RCC_CIER)*.

The LSE oscillator is switched ON and OFF using the LSEON bit. The LSE remains enabled when the system enters Stop or Standby mode.

In addition, the LSE clock can be driven to the MCO1 output and used as clock source for other application components.

The LSE also offers a programmable driving capability (LSEDRV[1:0]) that can be used to modulate the amplifier driving capability. This driving capability must be chosen according to the external crystal/ceramic component requirement to insure a stable oscillation. The driving capability must be set before enabling the LSE oscillator.

HSI oscillator

The HSI block provides the default clock to the product.

The HSI is a high-speed internal RC oscillator which can be used directly as system clock, peripheral clock, or as PLL input. A predivider allows the application to select an HSI output frequency of 8, 16, 32 or 64 MHz. This predivider is controlled by the HSIDIV.

The HSI advantages are the following:

- Low-cost clock source since no external crystal is required
- Faster startup time than HSE (a few microseconds)

The HSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The HSI can be switched ON and OFF using the HSION bit. Note that the HSI cannot be switched OFF if one of the two conditions is met:

- The HSI is used directly (via software mux) as system clock
- The HSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSION bit to 0.

Note that the HSIDIV cannot be changed if the HSI is selected as reference clock for at least one enabled PLL (PLLxON bit set to 1). In that case the hardware does not update the HSIDIV with the new value. However it is possible to change the HSIDIV if the HSI is used directly as system clock.

The HSIRDY flag indicates if the HSI is stable or not. At startup, the HSI output clock is not released until this bit is set by hardware.

The HSI clock can also be used as a backup source (auxiliary clock) if the HSE fails (refer to *Section : CSS on HSE*). The HSI can be disabled or not when the system enters Stop mode, please refer to *Section 8.5.7: Handling clock generators in Stop and Standby mode* for additional information.

In addition, the HSI clock can be driven to the MCO1 output and used as clock source for other application components.

Care must be taken when the HSI is used as kernel clock for communication peripherals, the application must take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available,
- the frequency accuracy.

Note: The HSI can remain enabled when the system is in Stop mode (see Section 8.5.7 for additional information).

HSION, HSIRDY and HSIDIV bits are located in the RCC source control register (RCC_CR).

HSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations. That is why each device is factory calibrated by STMicroelectronics to improve accuracy (refer to the product datasheet for more information).

After a power-on reset, the factory calibration value is loaded in the HSICAL[11:0] bits.

If the application is subject to voltage or temperature variations, this may affect the RC oscillator frequency. The user application can trim the HSI frequency using the HSITRIM bits.

CSI oscillator

The CSI is a low-power RC oscillator which can be used directly as system clock, peripheral clock, or PLL input.

The CSI advantages are the following:

- Low-cost clock source since no external crystal is required
- Faster startup time than HSE (a few microseconds)
- Very low-power consumption,

The CSI provides a clock frequency of about 4 MHz, while the HSI is able to provide a clock up to 64 MHz.

CSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The CSI can be switched ON and OFF through the CSION bit. The CSIRDY flag indicates whether the CSI is stable or not. At startup, the CSI output clock is not released until this bit is set by hardware.

The CSI cannot be switched OFF if one of the two conditions is met:

- The CSI is used directly (via software mux) as system clock
- The CSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the CSION bit to 0.

The CSI can be disabled or not when the system enters Stop mode (refer to *Section 8.5.7: Handling clock generators in Stop and Standby mode* for additional information).

In addition, the CSI clock can be driven to the MCO2 output and used as clock source for other application components.

Even if the CSI settling time is faster than the HSI, care must be taken when the CSI is used as kernel clock for communication peripherals: the application has to take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available,
- the frequency precision.

CSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations, this is why each device is factory calibrated by STMicroelectronics to achieve improve (refer to the product datasheet for more information).

After reset, the factory calibration value is loaded in the CSICAL[7:0] bits.

If the application is subject to voltage or temperature variations, this may affect the RC oscillator frequency. The user application can trim the CSI frequency using the CSITRIM bits.

HSI48 oscillator

The HSI48 is an RC oscillator that delivers a 48 MHz clock that can be used directly as kernel clock for some peripherals.

The HSI48 oscillator mainly aims at providing a high precision clock to the USB peripheral by means of a special Clock Recovery System (CRS) circuitry, which could use the USB SOF signal, the LSE, or an external signal, to automatically adjust the oscillator frequency on-the-fly, in very small granularity.

The HSI48 oscillator is disabled as soon as the system enters Stop or Standby mode. When the CRS is not used, this oscillator is free running and thus subject to manufacturing process variations. That is why each device is factory calibrated by STMicroelectronics to achieve an accuracy of ACC_{HSI48} (refer to the product datasheet of the for more information).

For more details on how to configure and use the CRS, please refer to *Section 9: Clock recovery system (CRS)*.

The HSI48RDY flag indicates whether the HSI48 oscillator is stable or not. At startup, the HSI48 output clock is not released until this bit is set by hardware.

The HSI48 can be switched ON and OFF using the HSI48ON bit.

The HSI48 clock can also be driven to the MCO1 multiplexer and used as clock source for other application components.

Note: HSI48ON and HSI48RDY bits are located in the RCC source control register (RCC_CR).

LSI oscillator

The LSI acts as a low-power clock source that can be kept running when the system is in Stop or Standby mode for the independent watchdog (IWDG) and Auto-Wakeup Unit (AWU). The clock frequency is around 32 kHz. For more details, refer to the electrical characteristics section of the datasheet.

The LSI can be switched ON and OFF using the LSION bit. The LSIRDY flag indicates whether the LSI oscillator is stable or not. If an independent watchdog is started either by hardware or software, the LSI is forced ON and cannot be disabled.

The LSI remains enabled when the system enters Stop or Standby mode (refer to *Section 8.5.7: Handling clock generators in Stop and Standby mode* for additional information).

At LSI startup, the clock is not provided until the hardware sets the LSIRDY bit. An interrupt can be generated if enabled in the *RCC clock source interrupt enable register (RCC_CIER)*.

In addition, the LSI clock can be driven to the MCO2 output and used as a clock source for other application components.

Note: Bits LSION and LSIRDY are located into the RCC Clock Control and Status Register (RCC_CSR).

8.5.3 Clock Security System (CSS)

CSS on HSE

The clock security system can be enabled by software via the HSECSSON bit. The HSECSSON bit can be enabled even when the HSEON is set to 0.

The CSS on HSE is enabled by the hardware when the HSE is enabled and ready, and HSECSSON set to 1.

The CSS on HSE is disabled when the HSE is disabled. As a result, this function does not work when the system is in Stop mode.

It is not possible to clear directly the HSECSSON bit by software.

The HSECSSON bit is cleared by hardware when a system reset occurs or when the system enters Standby mode (see *Section 8.4.2: System reset*).

If a failure is detected on the HSE clock, the system automatically switches to the HSI in order to provide a safe clock. The HSE is then automatically disabled, a clock failure event is sent to the break inputs of advanced-control timers (TIM1, TIM8, TIM15, TIM16, and TIM17), and an interrupt is generated to inform the software about the failure (CSS interrupt: ***rcc_hsecss_it***), thus allowing the MCU to perform rescue operations. If the HSE output was used as clock source for PLLs when the failure occurred, the PLLs are also disabled.

If an HSE clock failure occurs when the CSS is enabled, the CSS generates an interrupt which causes the automatic generation of an NMI. The HSECSSF flag in *RCC clock source interrupt flag register (RCC_CIFR)* is set to 1 to allow the application to identify the failure source. The NMI routine is executed indefinitely until the HSECSSF bit is cleared. As a consequence, the application has to clear the HSECSSF flag in the NMI ISR by setting the HSECSSC bit in the *RCC clock source interrupt clear register (RCC_CICR)*.

CSS on LSE

A clock security system on the LSE oscillator can be enabled by software by programming the LSECSSON bit in the *RCC backup domain control register (RCC_BDCR)*.

This bit can be disabled only by hardware when the following conditions are met:

- after a pwr_vsw_rst (V_{SW} software reset)
- or after a failure detection on LSE.

LSECSSON bit must be written after the LSE is enabled (LSEON bit set by software) and ready (LSERDY set by hardware), and after the RTC clock has been selected through the RTCSEL bit.

The CSS on LSE works in all modes (Run, Stop and Standby) except VBAT.

If an LSE failure is detected, the LSE clock is no more delivered to the RTC but the value of RTCSEL, LSECSSON and LSEON bits are not changed by the hardware.

A wakeup is generated in Standby mode. In other modes an interrupt (**rcc_lsecss_it**) can be sent to wake up the software. The software must then disable the LSECSSON bit, stop the defective LSE (clear LSEON bit), and can change the RTC clock source (no clock or LSI or HSE) through RTCSEL bits, or take any required action to secure the application.

8.5.4 Clock output generation (MCO1/MCO2)

Two micro-controller clock output (MCO) pins, MCO1 and MCO2, are available. A clock source can be selected for each output.The selected clock can be divided thanks to configurable prescaler (refer to *Figure 45* for additional information on signal selection).

MCO1 and MCO2 outputs are controlled via MCO1PRE[3:0], MCO1[2:0], MCO2PRE[3:0] and MCO2[2:0] located in the *RCC clock configuration register (RCC_CFGR)*.

The GPIO port corresponding to each MCO pin, has to be programmed in alternate function mode.

The clock provided to the MCOs outputs must not exceed the maximum pin speed (refer to the product datasheet for information on the supported pin speed).

8.5.5 PLL description

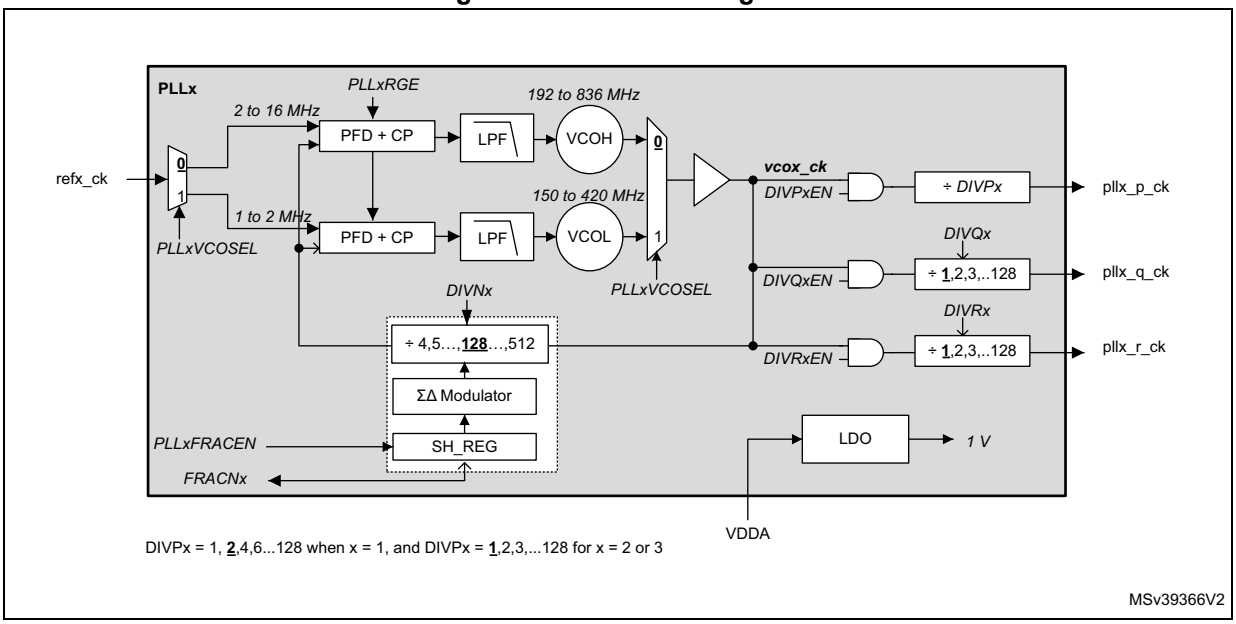
The RCC features three PLLs:

- A main PLL, PLL1, which is generally used to provide clocks to the CPU and to some peripherals.
- Two dedicated PLLs, PLL2 and PLL3, which are used to generate the kernel clock for peripherals.

The PLLs integrated into the RCC are completely independent. They offer the following features:

- Two embedded VCOs:
  - A wide-range VCO (VCOH)
  - A low-frequency VCO (VCOL)
- Input frequency range:
  - 1 to 2 MHz when VCOL is used
  - 2 to 16 MHz when VCOH is used
- Capability to work either in integer or Fractional mode
- 13-bit Sigma-Delta modulator, allowing to fine-tune the VCO frequency by steps of 11 to 0.3 ppm.
- The Sigma-Delta modulator can be updated on-the-fly, without generating frequency overshoots on PLLs outputs.
- Each PLL offer 3 outputs with post-dividers

Figure 47. PLL block diagram



The PLLs are controlled via RCC_PLLxDIVR, RCC_PLLxFRACR, RCC_PLLCFGR and RCC_CR registers.

The frequency of the reference clock provided to the PLLs (**refx_ck**) must range from 1 to 16 MHz. The user application has to program properly the DIVMx dividers of the *RCC PLLs clock source selection register (RCC_PLLCKSELR)* in order to match this condition. In addition, the PLLxRGE of the *RCC PLLs Configuration Register (RCC_PLLCFGR)* field must be set according to the reference input frequency to guarantee an optimal performance of the PLL.

The user application can then configure the proper VCO: if the frequency of the reference clock is lower or equal to 2 MHz, then VCOL must be selected. Otherwise for a frequency equal or higher to 2 MHz, VCOH must be chosen. To reduce the power consumption, it is recommended to configure the VCO output to the lowest frequency.

DIVNx loop divider has to be programmed to achieve the expected frequency at VCO output. In addition, the VCO output range must be respected.

The PLLs operate in integer mode when the value of SH_REG (FRACNx shadow register) is set to 0. The SH_REG is updated with the FRACNx value when PLLxFRACEN bit goes from 0 to 1. The Sigma-Delta modulator is designed in order to minimize the jitter impact while allowing very small frequency steps.

The PLLs can be enabled by setting PLLxON to 1. The bits PLLxRDY indicate that the PLL is ready (i.e. locked).

*Note: Before enabling the PLLs, make sure that the reference frequency (**refx_ck**) provided to the PLL is stable, so the hardware does not allow changing DIVMx when the PLLx is ON and it is also not possible to change PLLSRC when one of the PLL is ON.*

The hardware prevents writing PLL1ON to 0 if the PLL1 is currently used to deliver the system clock. There are other hardware protections on the clock generators (refer to sections HSE oscillator, HSI oscillator and CSI oscillator).

The following PLL parameters cannot be changed once the PLL is enabled: DIVNx, PLLxRGE, PLLxVCOSEL, DIVPx, DIVQx, DIVRx, DIVPxEN, DIVQxEN and DIVRxEN.

To insure an optimal behavior of the PLL when one of the post-divider (DIVP, DIVQ or DIVR) is not used, the application shall set the enable bit (DIVyEN) as well as the corresponding post-divider bits (DIVP, DIVQ or DIVR) to 0.

If the above rules are not respected, the PLL output frequency is not guaranteed.

Output frequency computation

When the PLL is configured in integer mode (SH_REG = 0), the VCO frequency (F_{VCO}) is given by the following expression:

$$F_{VCO} = F_{REF_CK} \times DIVN$$

$$F_{PLL_y_CK} = (F_{VCO} / (DIVy + 1)) \text{ with y = P, Q or R}$$

When the PLL is configured in fractional mode (SH_REG different from 0), the DIVN divider must be initialized before enabling the PLLs. However, it is possible to change the value of FRACNx on-the-fly without disturbing the PLL output.

This feature can be used either to generate a specific frequency from any crystal value with a good accuracy, or to fine-tune the frequency on-the-fly.

For each PLL, the VCO frequency is given by the following formula:

$$F_{VCO} = F_{ref_ck} \times \left(DIVN + \frac{FRACN}{2^{(13)}} \right)$$

Note: For PLL1, DIVP bitfield can only take odd values or 1 (no division).

The PLLs are disabled by hardware when:

- The system enters Stop or Standby mode.
- An HSE failure occurs when HSE or PLL (clocked by HSE) are used as system clock.

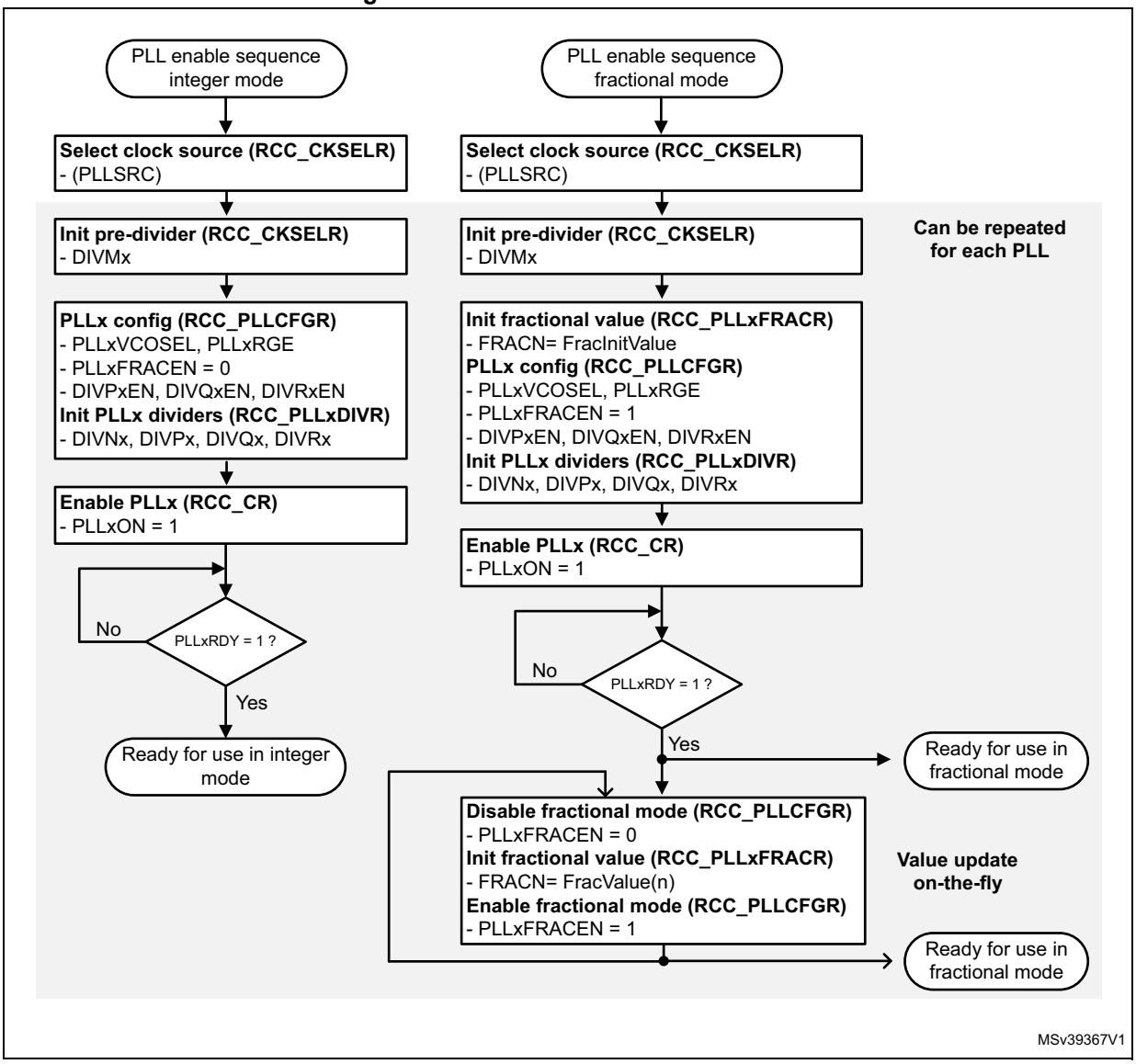
PLL initialization phase

Figure 48 shows the recommended PLL initialization sequence in integer and fractional mode. The PLLx are supposed to be disabled at the start of the initialization sequence:

1. Initialize the PLLs registers according to the required frequency.
   – Set PLLxFRACEN of *RCC PLLs Configuration Register (RCC_PLLCFGR)* to 0 for integer mode.
   – For fractional mode, set FRACN to the required initial value (FracInitValue) and then set PLLxFRACEN to 1.
2. Once the PLLxON bit is set to 1, the user application has to wait until PLLxRDY bit is set to 1. If the PLLx is in fractional mode, the PLLxFRACEN bit must not be set back to 0 as long as PLLxRDY = 0.
3. Once the PLLxRDY bit is set to 1, the PLLx is ready to be used.
4. If the application intends to tune the PLLx frequency on-the-fly (possible only in fractional mode), then:
   a) PLLxFRACEN must be set to 0,
   When PLLxFRACEN = 0, the Sigma-Delta modulator is still operating with the value latched into SH_REG.
   b) A new value must be uploaded into PLLxFRACR (FracValue(n)).
   c) PLLxFRACEN must be set to 1, in order to latch the content of PLLxFRACR into its shadow register.

Note: When the PLLxRDY goes to 1, it means that the difference between the PLLx output frequency and the target value is lower than ±2%.

Figure 48. PLLs Initialization Flowchart



8.5.6 System clock (sys_ck)

System clock selection

After a system reset, the HSI is selected as system clock and all PLLs are switched OFF. When a clock source is used for the system clock, it is not possible for the software to disable the selected source via the xxxON bits.

Of course, the system clock can be stopped by the hardware when the System enters Stop or Standby mode.

When the system is running, the user application can select the system clock (**sys_ck**) among the 4 following sources:

- HSE
- HSI
- CSI or
- pll1_p_ck

This function is controlled by programming the *RCC clock configuration register (RCC_CFGR)*. A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source that is not yet ready is selected, the switch occurs when the clock source is ready.
The SWS status bits in the *RCC clock configuration register (RCC_CFGR)* indicate which clock is currently used as system clock. The other status bits in the RCC_CR register indicate which clock(s) is (are) ready.

System clock generation

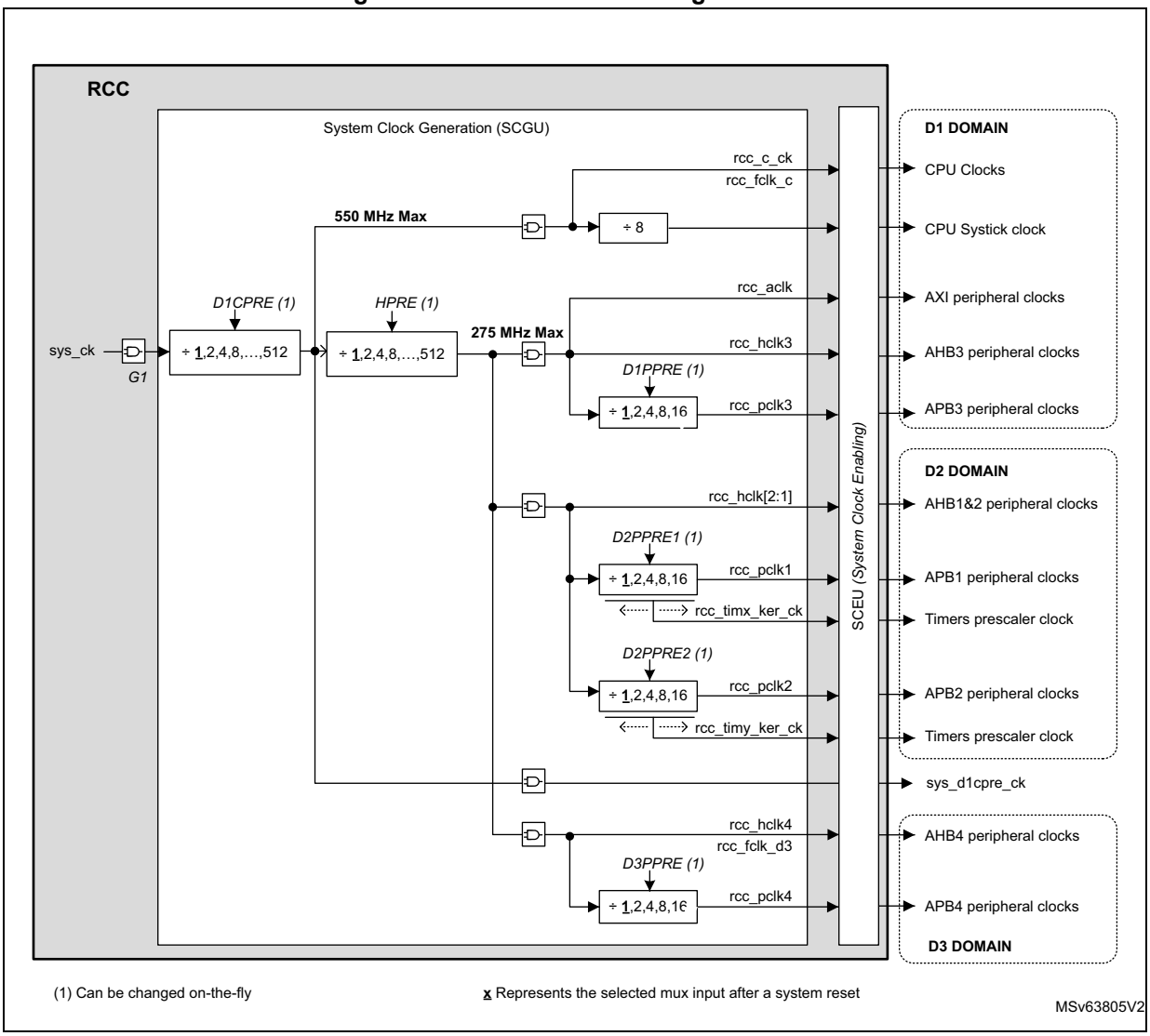
Figure 49 shows a simplified view of the clock distribution for the CPU and busses. All the dividers shown in the block diagram can be changed on-the-fly without generating timing violations. This feature is a very simply solution to adapt the busses frequencies to the application needs, thus optimizing the power consumption.

The D1CPRE divider can be used to adjust the CPU clock. However this also impacts the clock frequency of all bus matrix.

In the same way, HPRE divider can be used to adjust the clock for D1 domain bus matrix, but this also impacts the clock frequency of bus matrix of D2 and D3 domains.

Most of the prescalers are controlled via RCC_D1CFGR, RCC_D2CFGR and RCC_D3CFGR registers.

Figure 49. Core and bus clock generation



This block also provides the clock for the timers (**rcc_timx_ker_ck and rcc_timy_ker_ck**). The frequency of the timers clock depends on the APB prescaler corresponding to the bus to which the timer is connected, and on TIMPRE bit. *Table 53* shows how to select the timer clock frequency.

Table 53. Ratio between clock timer and pclk

D2PPRE1 (1) D2PPRE2	TIMPRE (2)		$F_{rcc_timx_ker_ck}$ $F_{rcc_timy_ker_ck}$	F_{rcc_pclk1} F_{rcc_pclk2}	Comments
0xx	0	→	F_{rcc_hclk1}	F_{rcc_hclk1}	The timer clock is equal to the bus clock.
100	0	→	F_{rcc_hclk1}	F_{rcc_hclk1} / 2	The timer clock is twice as fast as the bus clock.
101	0	→	F_{rcc_hclk1} / 2	F_{rcc_hclk1} / 4	
110	0	→	F_{rcc_hclk1} / 4	F_{rcc_hclk1} / 8	
111	0	→	F_{rcc_hclk1} / 8	F_{rcc_hclk1} / 16	

Table 53. Ratio between clock timer and pclk (continued)

D2PPRE1 (1) D2PPRE2	TIMPRE (2)		$F_{rcc_timx_ker_ck}$ $F_{rcc_timy_ker_ck}$	F_{rcc_pclk1} F_{rcc_pclk2}	Comments
0xx	1	→	F_{rcc_hclk1}	F_{rcc_hclk1}	The timer clock is equal to the bus clock.
100	1	→	F_{rcc_hclk1}	F_{rcc_hclk1} / 2	The timer clock is twice as fast as the bus clock.
101	1	→	F_{rcc_hclk1}	F_{rcc_hclk1} / 4	The timer clock is 4 times faster than the bus clock.
110	1	→	F_{rcc_hclk1} / 2	F_{rcc_hclk1} / 8	
111	1	→	F_{rcc_hclk1} / 4	F_{rcc_hclk1} / 16	

1. D2PPRE1 and D2PPRE2 belong to *RCC domain 2 clock configuration register (RCC_D2CFGR)*.
2. TIMPRE belongs to *RCC clock configuration register (RCC_CFGR)*.

8.5.7 Handling clock generators in Stop and Standby mode

When the whole system enters Stop mode, all the clocks (system and kernel clocks) are stopped as well as the following clock sources:

- CSI, HSI (depending on HSIKERON, and CSIKERON bits)
- HSE
- PLL1, PLL2 and PLL3
- HSI48

The content of the RCC registers is not altered except for PLL1ON, PLL2ON, PLL3ON HSEON and HSI48ON which are set to 0.

Exiting Stop mode

When the microcontroller exits system Stop mode via a wake-up event, the application can select which oscillator (HSI and/or CSI) will be used to restart. The STOPWUCK bit selects the oscillator used as system clock. The STOPKERWUCK bit selects the oscillator used as kernel clock for peripherals. The STOPKERWUCK bit is useful if after a system Stop a peripheral needs a kernel clock generated by an oscillator different from the one used for the system clock.

All these bits belong to the *RCC clock configuration register (RCC_CFGR)*. *Table 54* gives a detailed description of their behavior.

Table 54. STOPWUCK and STOPKERWUCK description

STOPWUCK	STOPKERWUCK		Activated oscillator when the system exits Stop mode	Distributed clocks when System exits Stop mode	
				System Clock	Kernel Clock
0	0	→	HSI	HSI	HSI
	1	→	HSI and CSI		HSI and/or CSI
1	0	→		CSI	
	1	→	CSI		CSI

During Stop mode

There are two specific cases where the HSI or CSI can be enabled during system Stop mode:

- When a dedicated peripheral requests the kernel clock:

  In this case the peripheral will receive the HSI or CSI according to the kernel clock source selected for this peripheral (via PERxSRC).
- When the bits HSIKERON or CSIKERON are set:

  In this case the HSI and CSI are kept running during Stop mode but the outputs are gated. In that way, the clock will be available immediately when the system exits Stop mode or when a peripheral requests the kernel clock (see *Table 55* for details).

HSIKERON and CSIKERON bits belong to *RCC source control register (RCC_CR)*. *Table 55* gives a detailed description of their behavior.

Table 55. HSIKERON and CSIKERON behavior

HSIKERON (CSIKERON)		HSI (CSI) state during Stop mode	HSI (CSI) Setting time
0	→	OFF	$t_{su(HSI)}$ ($t_{su(CSI)}$) (1)
1	→	Running and Gated	Immediate

1. $t_{su(HSI)}$ and $t_{su(CSI)}$ are the startup times of the HSI and CSI oscillators (see refer to the product datasheet for the values of these parameters).

When the microcontroller exists system Standby mode, the HSI is selected as system and kernel clock, the RCC registers are reset to their initial values except for the RCC_RSR (or RCC_C1_RSR) and RCC_BDCR registers.

Note as well that the HSI and CSI outputs provide two clock paths (see *Figure 45*):

- one path for the system clock (**hsi_ck** or **csi_ck**)
- one path for the peripheral kernel clock (**hsi_ker_ck** or **csi_ker_ck**).

When a peripheral requests the kernel clock in system Stop mode, only the path providing the **hsi_ker_ck** or **csi_ker_ck** is activated.

Caution: It is not guaranteed that the CPU will get automatically the same clock frequencies when leaving CStop mode: this mainly depends on the System state. For example If the CPU goes to CStop, while the D3 domain is kept in CRun, when the CPU exits from CStop, the clock settings remain unchanged. If the D3 domain goes to CStop while the CPU is also in CStop, then when the CPU exits from CStop, the CPU will operate with HSI or CSI when it left the CStop mode.

8.5.8 Kernel clock selection

Some peripherals are designed to work with two different clock domains that operate asynchronously:

- a clock domain synchronous with the register and bus interface (**ckg_bus_perx** clock)
- and a clock domain generally synchronous with the peripheral (kernel clock).

The benefit of having peripherals supporting these two clock domains is that the user application has more freedom to choose optimized clock frequency for the CPU, bus matrix and for the kernel part of the peripheral.

As a consequence, the user application can change the bus frequency without reprogramming the peripherals. As an example an on-going transfer with UART will not be disturbed if its APB clock is changed on-the-fly.

Table 56 shows the kernel clock that the RCC can deliver to the peripherals. Each row of *Table 56* represents a MUX and the peripherals connected to its output. The columns starting from number 5 represents the clock sources. Column 3 gives the maximum allowed frequency at each MUX output. It is up to the user to respect these requirements.

Table 56. Kernel clock distribution overview

Peripherals	Clock mux control bits	Maximum allowed frequency [MHz]				Domain	Clock sources																						
		VOS0	VOS1	VOS2	VOS3		pll1_q_ck	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck	pll3_q_ck	pll3_r_ck	sys_ck	bus clocks (1)	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsi_ck	per_ck (2)	I2S_CKIN	USB_PHY1	spdifrx_symb_ck	Disabled
LTDC	-	166	100	75	50	D1							x																
FMC	FMCSEL	275	200	150	100		1			2					0							3							
OCTOSPI	OCTOSPISEL	333	200	150	100		1			2					0							3							
SDMMC1	SDMMCSEL	250(4)	200(4)	150(4)	100(4)		0			1																			
SDMMC2		250	200	150	100	D2																							
DFSDM1 Aclk	SAI1SEL	250	200	150	100		0	1			2											4	3						
DFSDM1 clk	DFSDM1SEL	275	200	150	85									1	0														
FDCAN	FDCANSEL	125	100	75	50		1		2							0													
HDMI-CEC	CECSEL	1	1	1	1													2(3)		0	1								
I2C1,2,3, 5	I2C1235SEL	137.5	100	75	43								1		0		2	3											
LPTIM1	LPTIM1SEL	137.5	100	75	43			1					2		0					3	4	5							
TIM[8:1][17:12]	-	275	200	150	85										x														
RNG	RNGSEL	250	200	150	100		1												0	2	3								
SAI1	SAI1SEL	166	150	113	75		0	1			2											4	3						
SPDIFRX	SPDIFRXSEL	250	200	150	100		0			1			2				3												
SPI(I2S)1,2,3	SPI123SEL	250	180	125	75		0	1			2											4	3						

Table 56. Kernel clock distribution overview (continued)

Peripherals	Clock mux control bits	Maximum allowed frequency [MHz]				Domain	Clock sources																				
		VOS0	VOS1	VOS2	VOS3		pll1_q_ck	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck	pll3_q_ck	pll3_r_ck	sys_ck	bus clocks (1)	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsi_ck	per_ck (2)	I2S_CKIN	USB_PHY1	spdifrx_symb_ck	Disabled
SPI4,5	SPI45SEL	250	180	125	75	D2			1			2			0	5	3	4								
SWPMI	SWPMISEL	137.5	100	75	43										0		1									
USART1,6, 10 UART9	USART16910SEL	137.5	100	75	43				1			2			0		3	4		5						
USART2,3 UART4,5,7,8	USART234578SEL	137.5	100	75	43				1			2			0		3	4		5						
USB1OTG	USBSEL	125	100	50	50		1					2							3							0
USB1ULPI	-	60	60	45	40																		x			
ADC1,2, 3	ADCSEL	160(4)	160(4)	60	40(4)			0					1									2					
I2C4	I2C4SEL	137.5	100	75	43	D3							1		0		2	3								
LPTIM2	LPTIM2SEL	137.5	100	75	43			1					2		0					3	4	5				
LPTIM3,4,5	LPTIM345SEL	137.5	100	75	43			1					2		0					3	4	5				
LPUART1	LPUART1SEL	137.5	100	75	43				1			2			0		3	4		5						
SAI4_A	SAI4ASEL	166	150	113	75		0	1			2											4	3		5	
SAI4_B	SAI4BSEL	166	150	113	75		0	1			2											4	3		5	
SPI(I2S)6	SPI6SEL	250	180	125	75				1			2			0	5	3	4					6			
RTC/AWU	RTCSEL	1				VSW										3(5)				1	2					0

1. The bus clocks are the bus interface clocks to which the peripherals are connected, it can be APB, AHB or AXI clocks.
2. The per_ck clock could be hse_ck, hsi_ker_ck or csi_ker_ck according to CKPERSEL selection.
3. Clock CSI divided by 122.
4. With a duty cycle close to 50%, meaning that DIV[P/Q/R]x values shall be even. For SDMMCx, the duty cycle shall be 50% when supporting DDR.
5. Clock HSE divided by RTCPRE.

Figure 50 to *Figure 60* provide a more detailed description of kernel clock distribution. To simplify the drawings, the bus interface clocks (pclk, hclk) are not represented, even if they are gated with enable signals. Refer to *Section 8.5.11: Peripheral clock gating control* for more details.

To reduce the amount of switches, some peripherals share the same kernel clock source. Nevertheless, all peripherals have their dedicated enable signal.

Peripherals dedicated to audio applications

The audio peripherals generally need specific accurate frequencies, except for SPDIFRX. As shown in *Figure 50* and *Figure 51*, the kernel clock of the SAIs or SPI(I2S)s can be generated by:

- The PLL1 when the amount of active PLLs has to be reduced
- The PLL2 or 3 for optimal flexibility in frequency generation
- HSE, HSI or CSI for use-cases where the current consumption is critical
- I2S_CKIN when an external clock reference need to be used.

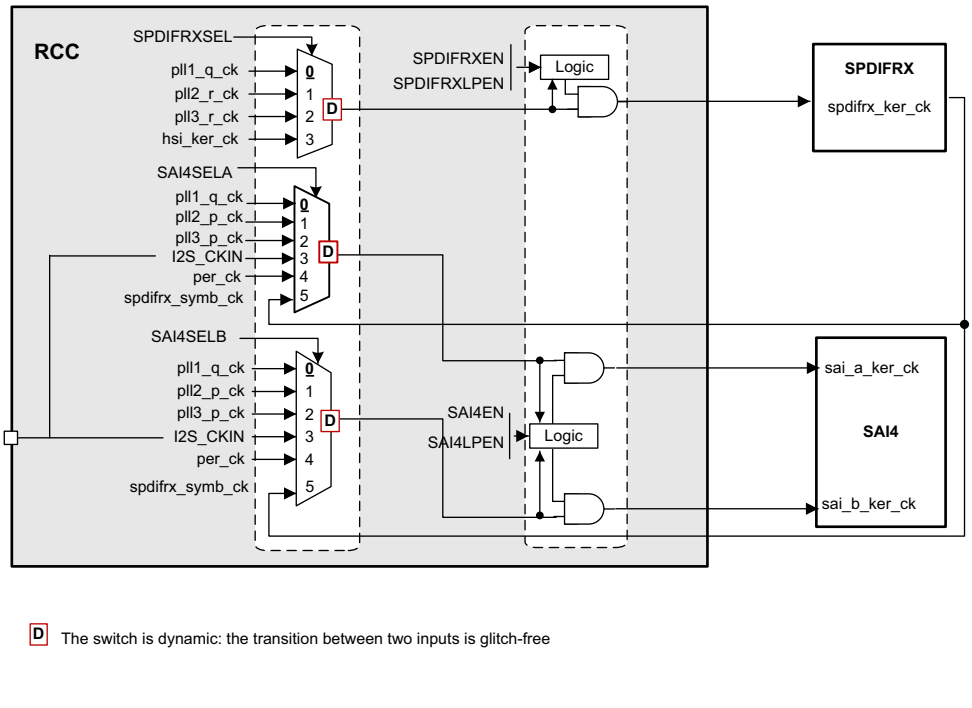
Note: The SPDIFRX does not require a specific frequency, but only a kernel clock frequency high enough to make the peripheral work properly. Refer to the SPDIFRX description for more details.

DFSDM1 can use the same clock as SAI1A. This is useful when DFSDM1 is used for audio applications.

To improve the flexibility, SAI4 can use different clock for each sub-block.

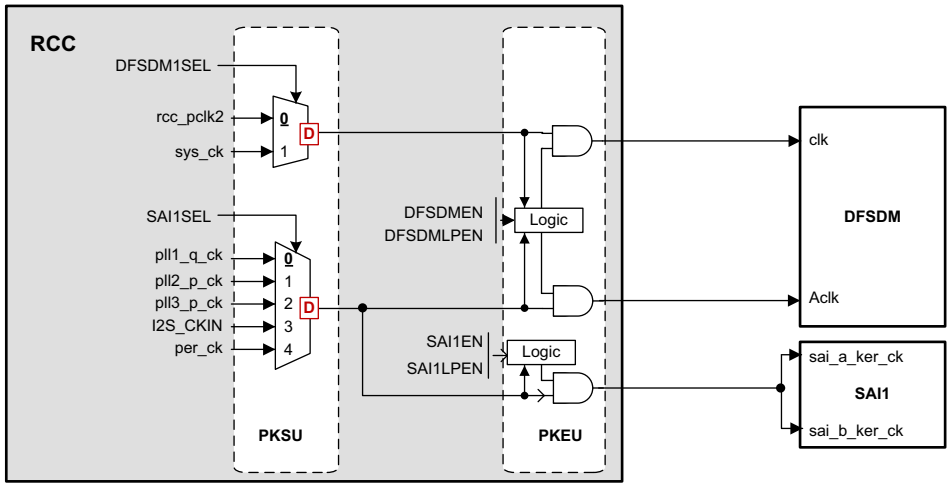
The SPI/I2S1, 2, and 3 share the same kernel clock source (see Figure 52).

Figure 50. Kernel clock distribution for SAIs and SPDIFRX



1. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripherals. For details on each enable cell, please refer to *Section 8.5.11: Peripheral clock gating control*.

Figure 51. Kernel clock distribution for DFSDM



1. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripherals. For details on each enable cell, please refer to *Section 8.5.11: Peripheral clock gating control*.

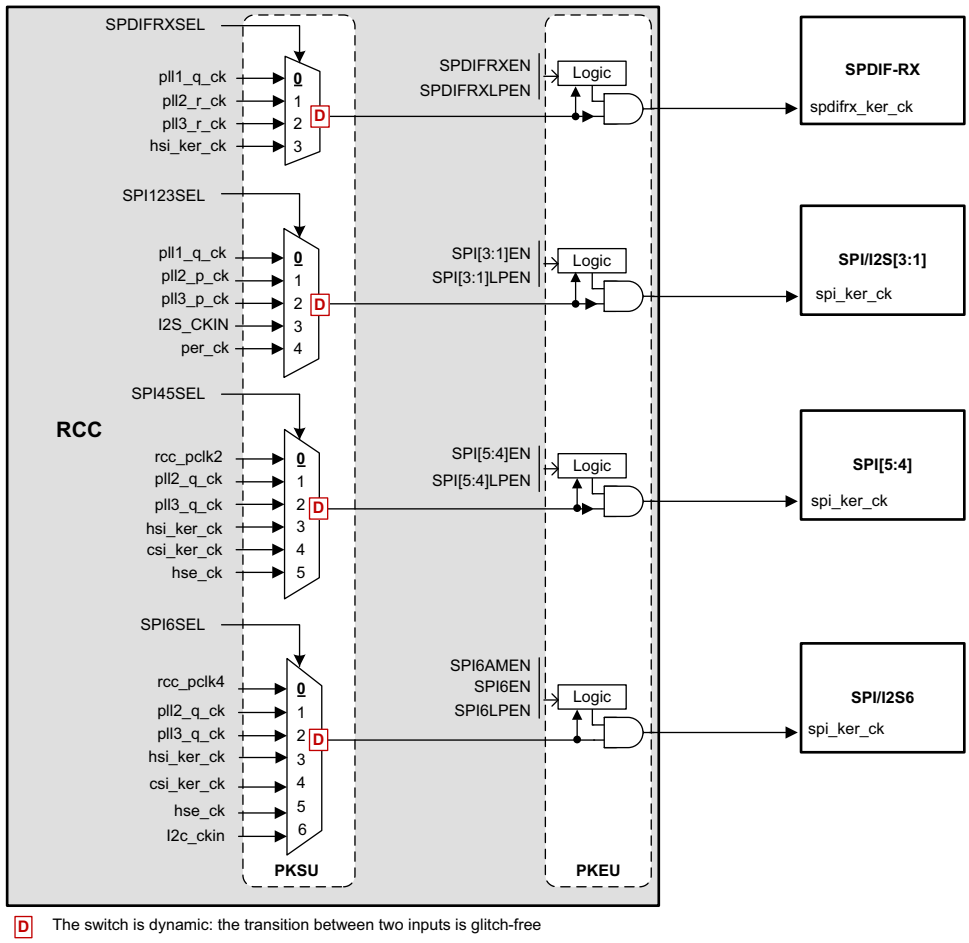
Peripherals dedicated to control and data transfer

Peripherals such as SPIs, I2Cs, UARTs do not need a specific kernel clock frequency but a clock fast enough to generate the correct baud rate, or the required bit clock on the serial interface. For that purpose the source can be selected among:

- PLL1 when the amount of active PLLs has to be reduced
- PLL2 or PLL3 if better flexibility is required. As an example, this solution allows changing the frequency bus via PLL1 without affecting the speed of some serial interfaces.
- HSI or CSI for low-power use-cases or when the peripheral has to quickly wake up from Stop mode (i.e. UART, I2C...).

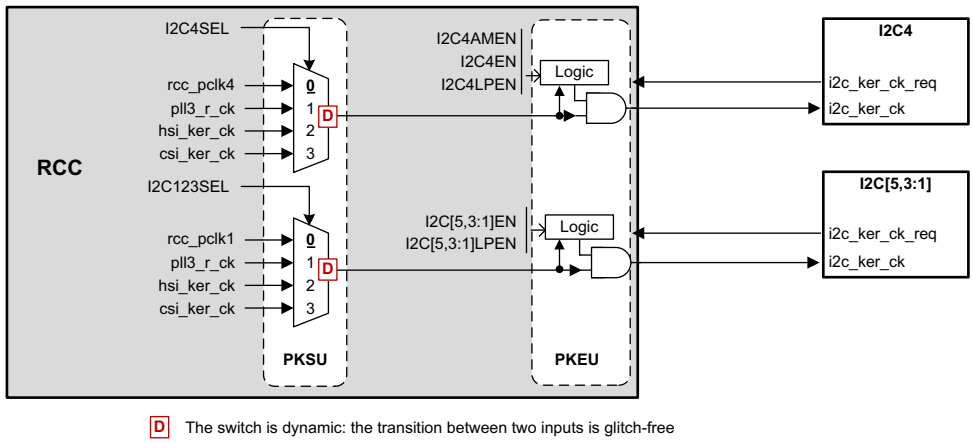
Note: UARTs also need the LSE clock when high baud rates are not required.

Figure 52. Kernel clock distribution for SPIs and SPI/I2S



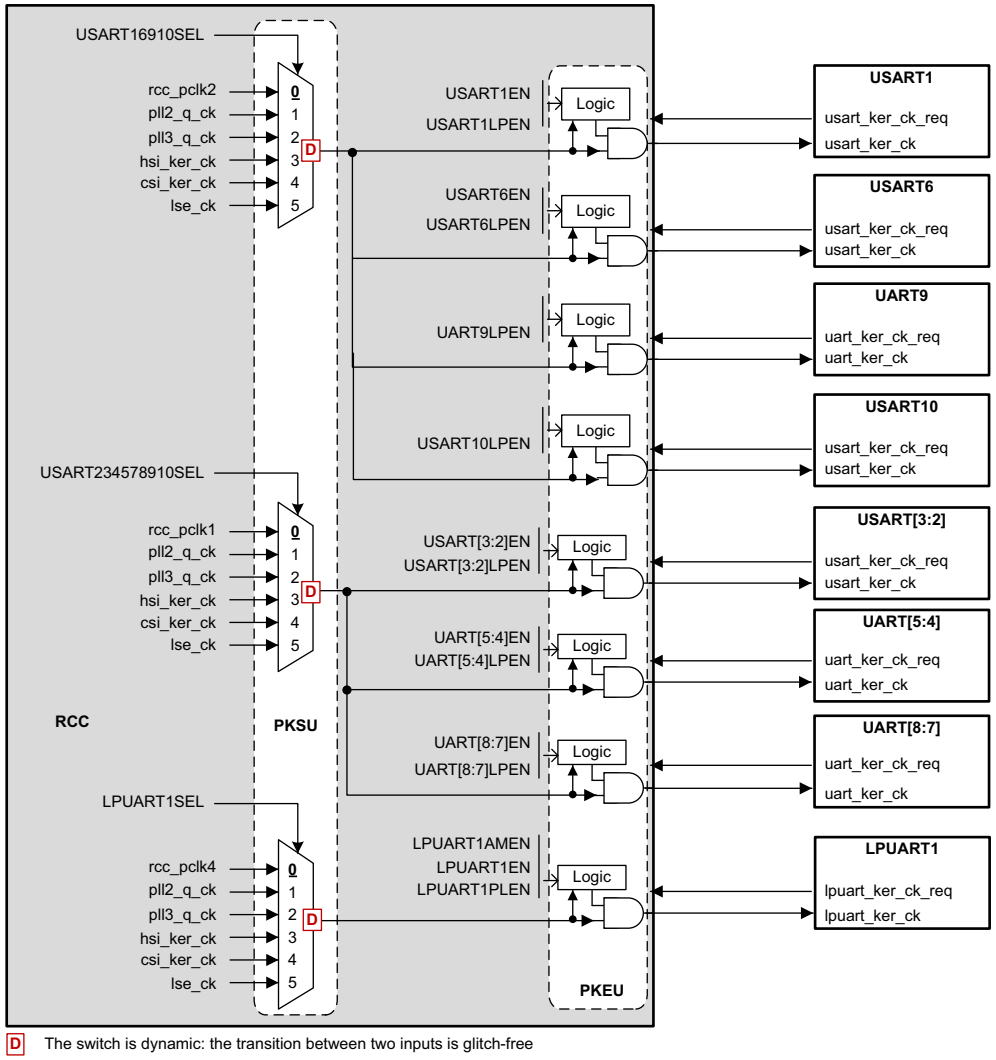
1. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. or details on each enable cell, please refer to *Section 8.5.11: Peripheral clock gating control*.

Figure 53. Kernel clock distribution for I2Cs



1. **X** represents the selected MUX input after a system reset
2. This figure does not show the connection of the bus interface clock to the peripheral, for details on each enable cell, please refer to *Section 8.5.11: Peripheral clock gating control*.

Figure 54. Kernel clock distribution for UARTs, USARTs and LPUART1



1. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral, for details on each enable cell, please refer to *Section 8.5.11: Peripheral clock gating control*.

Table 80. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x168	**RCC_C1_ AHB4LPENR**	Res.	Res.	SRAM4LPEN	BKPRAMLPEN	Res.	Res.	Res.	ADC3LPEN	Res.	Res.	BDMALPEN	Res.	CRCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKLPEN	GPIOJLPEN	Res.	GPIOHLPEN	GPIOGLPEN	GPIOFLPEN	GPIOELPEN	GPIODLPEN	GPIOCLPEN	GPIOBLPEN	GPIOALPEN
	Reset value			1	1				1			1		1									1	1		1	1	1	1	1	1	1	1
0x16C	**RCC_C1_ APB3LPENR**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WWDG1LPEN	Res.	Res.	LTDCLPEN	Res.	Res.	Res.
	Reset value																										1			1			
0x170	**RCC_C1_ APB1LLPENR**	UART8LPEN	UART7LPEN	DAC12LPEN	Res.	CECLPEN	Res.	I2C5LPEN	Res.	I2C3LPEN	I2C2LPEN	I2C1LPEN	UART5LPEN	UART4LPEN	USART3LPEN	USART2LPEN	SPDIFRXLPEN	SPI3LPEN	SPI2LPEN	Res.	Res.	Res.	Res.	LPTIM1LPEN	TIM14LPEN	TIM13LPEN	TIM12LPEN	TIM7LPEN	TIM6LPEN	TIM5LPEN	TIM4LPEN	TIM3LPEN	TIM2LPEN
	Reset value	1	1	1		1		1		1	1	1	1	1	1	1	1	1	1				1	1	1	1	1	1	1	1	1	1	1
0x174	**RCC_C1_ APB1HLPENR**	Res.	Res.	Res.	Res.	Res.	TIM24LPEN	TIM23LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	Res.	MDIOSLPEN	OPAMPLPEN	Res.	SWPMILPEN	CRSLPEN	Res.
	Reset value						1	1																	1			1	1		1	1	
0x178	**RCC_C1_ APB2LPENR**	Res.	DFSDM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI1LPEN	Res.	SPI5LPEN	Res.	TIM17LPEN	TIM16LPEN	TIM15LPEN	Res.	Res.	SPI4LPEN	SPI1LPEN	Res.	Res.	Res.	Res.	USART10LPEN	UART9LPEN	USART6LPEN	USART1LPEN	Res.	Res.	TIM8LPEN	TIM1LPEN
	Reset value		1								1		1		1	1	1			1	1					1	1	1	1			1	1
0x17C	**RCC_C1_ APB4LPENR**	Res.	Res.	Res.	Res.	Res.	DTSLPEN	Res.	Res.	Res.	Res.	SAI4LPEN	Res.	Res.	Res.	Res.	RTCAPBLPEN	VREFLPEN	COMP12LPEN	Res.	LPTIM5LPEN	LPTIM4LPEN	LPTIM3LPEN	LPTIM2LPEN	Res.	I2C4LPEN	Res.	SPI6LPEN	Res.	LPUART1LPEN	Res.	SYSCFGLPEN	Res.
	Reset value						1					1					1	1	1		1	1	1	1		1		1		1		1	
0x180 to 0x1FC	**reserved**	Reserved																															

Refer to *Section 2.3 on page 131* for the register boundary addresses.

Table 98. DMAMUX1, DMA1 and DMA2 connections(1) (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D2	APB2	DFSDM1	dfsdm1_dma0	dmamux1_req_in101	DMAMUX1	AHB1	D2	Requests
			dfsdm1_dma1	dmamux1_req_in102				
			dfsdm1_dma2	dmamux1_req_in103				
			dfsdm1_dma3	dmamux1_req_in104				
D2	APB2	TIM15	tim15_ch1_dma	dmamux1_req_in105				
			tim15_up_dma	dmamux1_req_in106				
			tim15_trig_dma	dmamux1_req_in107				
			tim15_com_dma	dmamux1_req_in108				
D2	APB2	TIM16	tim16_ch1_dma	dmamux1_req_in109				
			tim16_up_dma	dmamux1_req_in110				
D2	APB2	TIM17	tim17_ch1_mda	dmamux1_req_in111				
			tim17_up_dma	dmamux1_req_in112				
D3	AHB4	ADC3	adc3_dma	dmamux1_req_in115				
D2	APB2	UART9	uart9_rx_dma	dmamux1_req_in116				
			uart9_tx_dma	dmamux1_req_in117				
		USART10	usart10_rx_dma	dmamux1_req_in118				
			usart10_tx_dma	dmamux1_req_in119				
D2	AHB2	FMAC	fmac_rd_dma	dmamux1_req_in120				
			fmac_wr_dma	dmamux1_req_in121				
		CORDIC	cordic_rd_dma	dmamux1_req_in122				
			cordic_wr_dma	dmamux1_req_in123				
	APB1	I2C5	i2c5_rx_dma	dmamux1_req_in124				
			i2c5_tx_dma	dmamux1_req_in125				
		TIM23	tim23_ch1_dma	dmamux1_req_in126				
			tim23_ch2_dma	dmamux1_req_in127				
			tim23_ch3_dma	dmamux1_req_in128				
			tim23_ch4_dma	dmamux1_req_in129				
			tim23_up_dma	dmamux1_req_in130				
			tim23_trig_dma	dmamux1_req_in131				

13.3.3 DMAMUX2, BDMA (D3 domain)

Table 99. DMAMUX2 and BDMA connections

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	dmamux2 internal (Request generator)		dmamux2_req_in1	DMAMUX2	AHB4	D3	Requests
				dmamux2_req_in2				
				dmamux2_req_in3				
				dmamux2_req_in4				
				dmamux2_req_in5				
				dmamux2_req_in6				
				dmamux2_req_in7				
				dmamux2_req_in8				
D3	APB4	LPUART	dma_rx_lpuart	dmamux2_req_in9				
			dma_tx_lpuart	dmamux2_req_in10				
D3	APB4	SPI6	dma_rx_spi6	dmamux2_req_in11				
			dma_tx_spi6	dmamux2_req_in12				
D2	APB1	I2C4	dma_rx_i2c4	dmamux2_req_in13				
			dma_tx_i2c4	dmamux2_req_in14				
D3	APB4	SAI4	dma_a_sai4	dmamux2_req_in15				
			dma_b_sai4	dmamux2_req_in16				
D3	APB4	ADC3	dma_adc3	dmamux2_req_in17				

Inside the regular sequence, after each conversion is complete:

- The converted data are stored into the 16-bit ADC_DR register
- The EOC (end of regular conversion) flag is set
- An interrupt is generated if the EOCIE bit is set

Inside the injected sequence, after each conversion is complete:

- The converted data are stored into one of the four 16-bit ADC_JDRy registers
- The JEOC (end of injected conversion) flag is set
- An interrupt is generated if the JEOCIE bit is set

After the regular sequence is complete:

- The EOS (end of regular sequence) flag is set
- An interrupt is generated if the EOSIE bit is set

After the injected sequence is complete:

- The JEOS (end of injected sequence) flag is set
- An interrupt is generated if the JEOSIE bit is set

Then the ADC stops until a new external regular or injected trigger occurs or until bit ADSTART or JADSTART is set again.

Note: To convert a single channel, program a sequence with a length of 1.

29.4.14 Continuous conversion mode (CONT = 1)

This mode applies to regular channels only.

In Continuous conversion mode, when a software or hardware regular trigger event occurs, the ADC performs once all the regular conversions of the channels and then automatically restarts and continuously converts each conversions of the sequence. This mode is started with the CONT bit at 1 either by external trigger or by setting the ADSTART bit in the ADC_CR register.

Inside the regular sequence, after each conversion is complete:

- The converted data are stored into the 16-bit ADC_DR register
- The EOC (end of conversion) flag is set
- An interrupt is generated if the EOCIE bit is set

After the sequence of conversions is complete:

- The EOS (end of sequence) flag is set
- An interrupt is generated if the EOSIE bit is set

Then, a new sequence restarts immediately and the ADC continuously repeats the conversion sequence.

Note: To convert a single channel, program a sequence with a length of 1.

It is not possible to have both Discontinuous mode and Continuous mode enabled: it is forbidden to set both DISCEN = 1 and CONT = 1.

Injected channels cannot be converted continuously. The only exception is when an injected channel is configured to be converted automatically after regular channels in Continuous mode (using JAUTO bit), refer to Auto-injection mode section).

29.4.15 Starting conversions (ADSTART, JADSTART)

Software starts ADC regular conversions by setting ADSTART = 1.

When ADSTART is set, the conversion starts:

- Immediately: if EXTEN = 0x0 (software trigger)
- At the next active edge of the selected regular hardware trigger: if EXTEN is not equal to 0x0

Software starts ADC injected conversions by setting JADSTART = 1.

When JADSTART is set, the conversion starts:

- Immediately, if JEXTEN = 0x0 (software trigger)
- At the next active edge of the selected injected hardware trigger: if JEXTEN is not equal to 0x0

Note: In auto-injection mode (JAUTO = 1), use ADSTART bit to start the regular conversions followed by the auto-injected conversions (JADSTART must be kept cleared).

ADSTART and JADSTART also provide information on whether any ADC operation is currently ongoing. It is possible to re-configure the ADC while ADSTART = 0 and JADSTART = 0 are both true, indicating that the ADC is idle.

ADSTART is cleared by hardware:

- In Single mode with software regular trigger (CONT = 0, EXTSEL = 0x0)
  - At any end of regular conversion sequence (EOS assertion) or at any end of subgroup processing if DISCEN = 1
- In all cases (CONT = x, EXTSEL = x)
  - After execution of the ADSTP procedure asserted by the software.

Note: In Continuous mode (CONT = 1), ADSTART is not cleared by hardware with the assertion of EOS because the sequence is automatically relaunched.

When a hardware trigger is selected in Single mode (CONT = 0 and EXTSEL # 0x00), ADSTART is not cleared by hardware with the assertion of EOS to help the software which does not need to reset ADSTART again for the next hardware trigger event. This ensures that no further hardware triggers are missed.

JADSTART is cleared by hardware:

- In Single mode with software injected trigger (JEXTSEL = 0x0)
  - At any end of injected conversion sequence (JEOS assertion) or at any end of subgroup processing if JDISCEN = 1
- in all cases (JEXTSEL = x)
  - After execution of the JADSTP procedure asserted by the software.

Note: When the software trigger is selected, ADSTART bit should not be set if the EOC flag is still high.

60.10.17 SDMMC IDMA buffer size register (SDMMC_IDMABSIZER)

Address offset: 0x054

Reset value: 0x0000 0000

The SDMMC_IDMABSIZER register contains the buffers size when in double buffer configuration.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	IDMABNDT[7:0]								Res.	Res.	Res.	Res.	Res.
			rw	rw	rw	rw	rw	rw	rw	rw					

Bits 31:13 Reserved, must be kept at reset value.

Bits 12:5 **IDMABNDT[7:0]**: Number of bytes per buffer

This 8-bit value must be multiplied by 8 to get the size of the buffer in 32-bit words and by 32 to get the size of the buffer in bytes.

Example: IDMABNDT = 0x01: buffer size = 8 words = 32 bytes.

Example: IDMABNDT = 0x80: buffer size = 1024 words = 4 Kbytes

These bits can only be written by firmware when DPSM is inactive (DPSMACT = 0).

Bits 4:0 Reserved, must be kept at reset value.

60.10.18 SDMMC IDMA buffer 0 base address register (SDMMC_IDMABASE0R)

Address offset: 0x058

Reset value: 0x0000 0000

The SDMMC_IDMABASE0R register contains the memory buffer base address in single buffer configuration and the buffer 0 base address in double buffer configuration.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDMABASE0[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMABASE0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r

Bits 31:0 **IDMABASE0[31:0]**: Buffer 0 memory base address bits [31:2], must be word aligned (bit [1:0] are always 0 and read only)

This register can be written by firmware when DPSM is inactive (DPSMACT = 0), and can dynamically be written by firmware when DPSM active (DPSMACT = 1) and memory buffer 0 is inactive (IDMABACT = ‘1’).

60.10.19 SDMMC IDMA buffer 1 base address register (SDMMC_IDMABASE1R)

Address offset: 0x05C

Reset value: 0x0000 0000

The SDMMC_IDMABASE1R register contains the double buffer configuration second buffer memory base address.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDMABASE1[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDMABASE1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r

Bits 31:0 **IDMABASE1[31:0]**: Buffer 1 memory base address, must be word aligned (bit [1:0] are always 0 and read only)

This register can be written by firmware when DPSM is inactive (DPSMACT = 0), and can dynamically be written by firmware when DPSM active (DPSMACT = 1) and memory buffer 1 is inactive (IDMABACT = ‘0’).

60.10.20 SDMMC register map

The following table summarizes the SDMMC registers.

Table 498. SDMMC register map

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	**SDMMC_ POWER**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIRPOL	VSWITCHEN	VSWITCH	PWRCTRL[1:0]	
	Reset value																												0	0	0	0	0
0x004	**SDMMC_ CLKCR**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SELCLKRX[1:0]		BUSSPEED	DDR	HWFC_EN	NEGEDGE	WIDBUS[1:0]		Res.	PWRSAV	Res.	Res.	CLKDIV[9:0]									
	Reset value											0	0	0	0	0	0	0	0		0			0	0	0	0	0	0	0	0	0	0
0x008	**SDMMC_ ARGR**	CMDARG[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	**SDMMC_ CMDR**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDSUSPEND	BOOTEN	BOOTMODE	DTHOLD	CPSMEN	WAITPEND	WAITINT	WAITRESP[1:0]		CMDSTOP	CMDTRANS	CMDINDEX[5:0]					
	Reset value																																
0x010	**SDMMC_ RESPCMDR**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESPCMD[5:0]					
	Reset value																											0	0	0	0	0	0
0x014	**SDMMC_ RESP1R**	CARDSTATUS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018	**SDMMC_ RESP2R**	CARDSTATUS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x01C	**SDMMC_ RESP3R**	CARDSTATUS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x020	**SDMMC_ RESP4R**	CARDSTATUS[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x024	**SDMMC_ DTIMER**	DATATIME[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x028	**SDMMC_ DLENR**	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATALENGTH[24:0]																								
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

presets the counter to the value configured by FDCAN_TOCC.TOP and continues down-counting.

When the timeout counter is controlled by one of the FIFOs, an empty FIFO presets the counter to the value configured by FDCAN_TOCC.TOP. Down-counting is started when the first FIFO element is stored. Writing to FDCAN_TOCV has no effect.

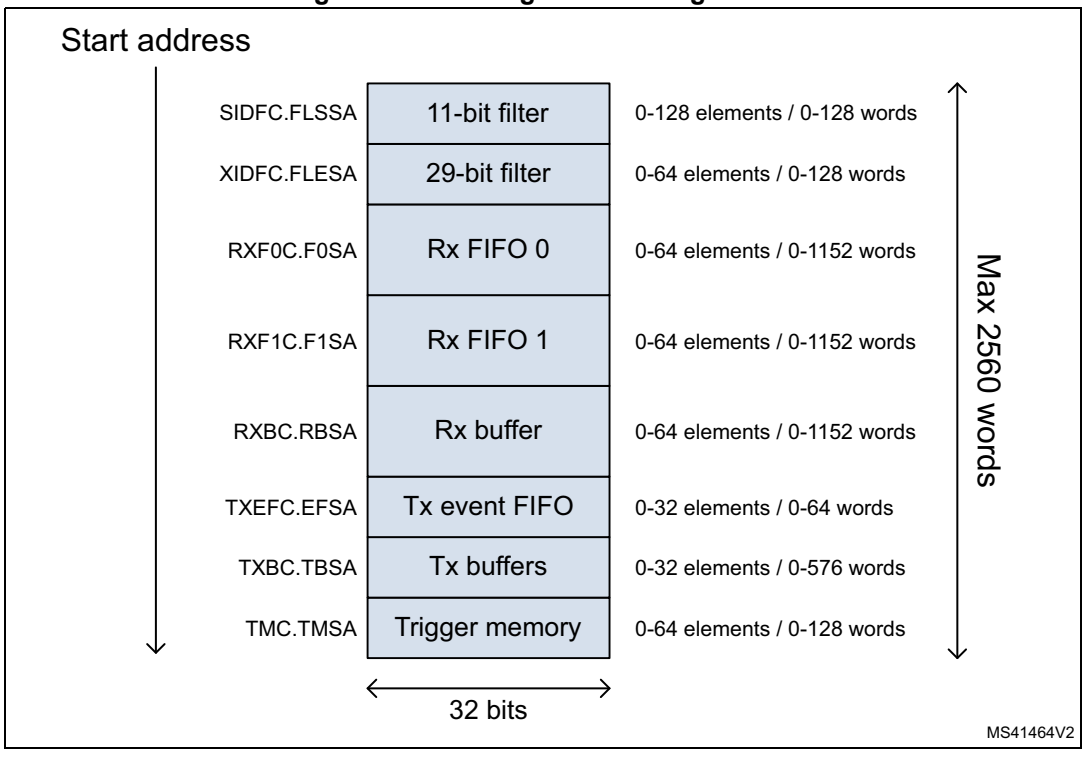
When the counter reaches 0, interrupt flag FDCAN_IR.TOO is set. In Continuous mode, the counter is immediately restarted at FDCAN_TOCC.TOP.

Note: The clock signal for the timeout counter is derived from the CAN core sample point signal. Therefore the point in time where the timeout counter is decremented may vary due to the synchronization/re-synchronization mechanism of the CAN core. If the baudrate switch feature in FDCAN is used, the timeout counter is clocked differently in arbitration and data fields.

61.4.2 Message RAM

The message RAM has a width of 32 bits. The FDCAN module can be configured to allocate up to 2560 words in the message RAM. It is not necessary to configure each of the sections listed in *Figure 767*, nor is there any restriction with respect to the sequence of the sections.

Figure 767. Message RAM configuration



When the FDCAN addresses the message RAM it addresses 32-bit words, not single bytes. The configured start addresses are 32-bit word addresses, i.e. only bits 15 to 2 are evaluated, the two least significant bits are ignored.

Note: The FDCAN does not check for erroneous configuration of the message RAM. Especially the configuration of the start addresses of the different sections and the number of elements of each section has to be done carefully to avoid falsification or loss of data.